

**Московский государственный технический
университет им. Н. Э. Баумана**

Курс «Технологии машинного обучения»

Отчёт по рубежному контролю №2

«Технологии разведочного анализа и обработки данных.»

Вариант № 9

Выполнил:
Ларкин Б. В.
группа ИУ5-61Б

Проверил:
Гапанюк Ю.Е.

Дата: 23.05.25

Дата:

Подпись:

Подпись:

Москва, 2025 г.

Задание:

Номер варианта: **9**

Номер набора данных, указанного в задаче: **1**

(<https://www.kaggle.com/datasets/atharvasoundankar/global-housing-market-analysis-2015-2024>)

Для студентов группы ИУ5-61Б – построить модели:

Линейная/логистическая регрессия, Случайный лес.

Задание

Для заданного набора данных (по Вашему варианту) постройте модели классификации или регрессии (в зависимости от конкретной задачи, рассматриваемой в наборе данных). Для построения моделей используйте методы 1 и 2 (по варианту для Вашей группы). Оцените качество моделей на основе подходящих метрик качества (не менее двух метрик). Какие метрики качества Вы использовали и почему? Какие выводы Вы можете сделать о качестве построенных моделей? Для построения моделей необходимо выполнить требуемую предобработку данных: заполнение пропусков, кодирование категориальных признаков, и т.д.

1. Введение

В рамках исследования был проведён анализ набора данных о динамике цен на жильё в различных странах мира за период с 2015 по 2024 год. Основной целью работы являлось построение моделей прогнозирования роста/падения цен на жильё с использованием методов машинного обучения.

2. Описание исходных данных

Исходный датасет содержит:

- 20 стран (США, Великобритания, Германия и др.)
- 9 временных периодов (ежегодные данные)

- 11 экономических показателей:
 - Индексы цен на жильё и аренду
 - Макроэкономические показатели (ВВП, инфляция, ставки по ипотеке)
 - Демографические данные (урбанизация, рост населения)

Особенности данных:

- Высокая вариативность между странами
- Ярко выраженная сезонность в некоторых регионах
- Наличие сложных нелинейных зависимостей

3. Ход выполнения:

Ход работы

Для решения этой задачи я построю модели регрессии, так как целевая переменная (цены на жильё) является непрерывной величиной. Использую два метода: линейную регрессию и случайный лес.

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
import seaborn as sns
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import classification_report
import matplotlib.pyplot as plt
from sklearn.preprocessing import LabelEncoder, OneHotEncoder, OrdinalEncoder
%matplotlib inline
```

Загрузка данных

```
# Загрузка данных
data = pd.read_csv('global_housing_market_extended.csv')
```

Основные характеристики датасета

```
# Первые 5 строк датасета
data.head()
```

	Country	Year	House Price Index	Rent Index	Affordability Ratio	Mortgage Rate (%)	Inflation Rate (%)	GDP Growth (%)	Population Growth (%)	Urbanization Rate (%)	Construction Index
0	USA	2015	117.454012	116.550001	9.587945	4.493292	1.514121	-0.752044	-0.796707	85.985284	118.089201
1	USA	2016	150.807258	51.440915	11.729189	5.662213	1.880204	-0.545400	-0.358084	69.127267	111.980515
2	USA	2017	123.194502	70.386040	8.506676	2.197469	2.398940	0.930895	0.596245	83.555279	85.973903
3	USA	2018	131.423444	91.469020	3.418054	4.537724	1.608407	-1.479587	2.321099	88.968961	134.671788
4	USA	2019	110.461377	56.837048	9.158097	3.700762	1.293249	1.961415	-0.879640	87.279612	90.702399

```
data.shape
```

(200, 11)

```
data.isnull().sum() #нет пропусков
```

```
Country      0
Year         0
House Price Index  0
Rent Index   0
Affordability Ratio  0
Mortgage Rate (%)  0
Inflation Rate (%)  0
GDP Growth (%)  0
Population Growth (%)  0
Urbanization Rate (%)  0
Construction Index  0
dtype: int64
```

Пропуски в данных отсутствуют.

Переведем категориальные признаки в численный вид.

```
def encode_country_feature(data, column='Country', method='label', drop_original=True, target_column=None):
    data = data.copy() # Чтобы избежать изменения исходного DataFrame

    if method == 'label':
        encoder = LabelEncoder()
        data[f'{column}_encoded'] = encoder.fit_transform(data[column])

    elif method == 'onehot':
        encoder = OneHotEncoder(sparse_output=False, handle_unknown='ignore')
        encoded = encoder.fit_transform(data[[column]])
        encoded_df = pd.DataFrame(encoded,
                                   columns=encoder.get_feature_names_out([column]),
                                   index=data.index)
        data = pd.concat([data, encoded_df], axis=1)

    elif method == 'ordinal':
        encoder = OrdinalEncoder()
        data[f'{column}_encoded'] = encoder.fit_transform(data[[column]])

    elif method == 'frequency':
        freq = data[column].value_counts(normalize=True)
        data[f'{column}_freq'] = data[column].map(freq)

    elif method == 'target':
        if target_column is None:
            raise ValueError("Для target encoding нужно задать target_column")
        mean_target = data.groupby(column)[target_column].mean()
        data[f'{column}_target'] = data[column].map(mean_target)

    else:
        raise ValueError(f"Неизвестный метод кодирования: {method}. "
                          f"Допустимые значения: 'label', 'onehot', 'ordinal', 'frequency', 'target'")

    if drop_original and method != 'onehot': # Для onehot исходная колонка не удаляется автоматически
        data.drop(columns=[column], inplace=True)

    return data

data = encode_country_feature(data.copy(), method='label')
```

51 ✓ 0.0s

Перекодируем год:

```
# Находим минимальный год в датасете
min_year = data['Year'].min()

# Преобразуем годы в year - min_year с помощью лямбда-функции
data['Year'] = data['Year'].apply(lambda x: x - min_year)
```

✓ 0.0s

Создадим новый целевой признак: предсказываем тренд

```
# Создание бинарной целевой переменной
data['Price_Up'] = (data.groupby('Country_encoded')['House Price Index'].pct_change() > 0).astype(int) # 1 если рост, 0 если падение
data = data.dropna() # Удаляем строки с NaN после pct_change
```

✓ 0.0s

Добавим новые признаки

```
data['Mortgage_Inflation_Ratio'] = data['Mortgage Rate (%)'] / (data['Inflation Rate (%)'] + 1e-6)
data['Urban_GDP_Interaction'] = data['Urbanization Rate (%)'] * data['GDP Growth (%)']
data['GDP_per_Capita'] = data['GDP Growth (%)'] / (data['Population Growth (%)'] + 1e-6)
data['Urban_Construction_Interaction'] = data['Urbanization Rate (%)'] * data['Construction Index']
data['Rent_ratio'] = data['House Price Index'] / data['Rent Index']
data['Affordability_r'] = (data['House Price Index'] - data['Rent Index']) / data['Affordability Ratio']

data['Mortgage_GDP_Interaction'] = data['Mortgage_Inflation_Ratio'] * data['GDP_per_Capita']
data['Rent_Affordability_Sq'] = data['Rent_ratio'] ** 2
data['log_Affordability_r'] = np.log1p(abs(data['Affordability_r']))

data=data.drop(columns=['Construction Index', 'Urbanization Rate (%)', 'Inflation Rate (%)', 'Mortgage Rate (%)', 'Population Growth (%)', 'House Price Index'])
```

✓ 0.0s

Экономическая интерпретация созданных признаков

1. Mortgage_Inflation_Ratio

= Ипотечная ставка / Инфляция

Экономический смысл: Показывает реальную стоимость кредита. При значении >1 кредиты становятся "дорогими", что снижает спрос на жильё.

2. Urban_GDP_Interaction

= Уровень урбанизации * Рост ВВП

Экономический смысл: В быстрорастущих урбанизированных регионах спрос на жильё растёт быстрее.

3. GDP_per_Capita

= Рост ВВП / Рост населения

Экономический смысл: Отражает улучшение благосостояния. При росте >1 повышается платежеспособный спрос.

4. Urban_Construction_Interaction

= Урбанизация * Индекс строительства

Экономический смысл: Показывает, успевает ли строительный сектор за миграцией в города.

5. Rent_ratio

= Цена жилья / Арендная ставка

Экономический смысл: Аналог P/E для недвижимости. Высокие значения (>20-30) сигнализируют о "перегреве" рынка.

6. Affordability_r

= (Цена жилья - Аренда) / Коэффициент доступности

Экономический смысл: Показывает, насколько выгоднее купить vs арендовать. Отрицательные значения - сигнал к коррекции цен.

7. Mortgage_GDP_Interaction

= Mortgage_Inflation_Ratio * GDP_per_Capita

Экономический смысл: Учитывает совместное влияние кредитных условий и благосостояния.

8. Rent_Affordability_Sq

= (Rent_ratio)^2

Экономический смысл: Усиливает влияние экстремальных значений арендной доходности.

9. Log_Affordability_r

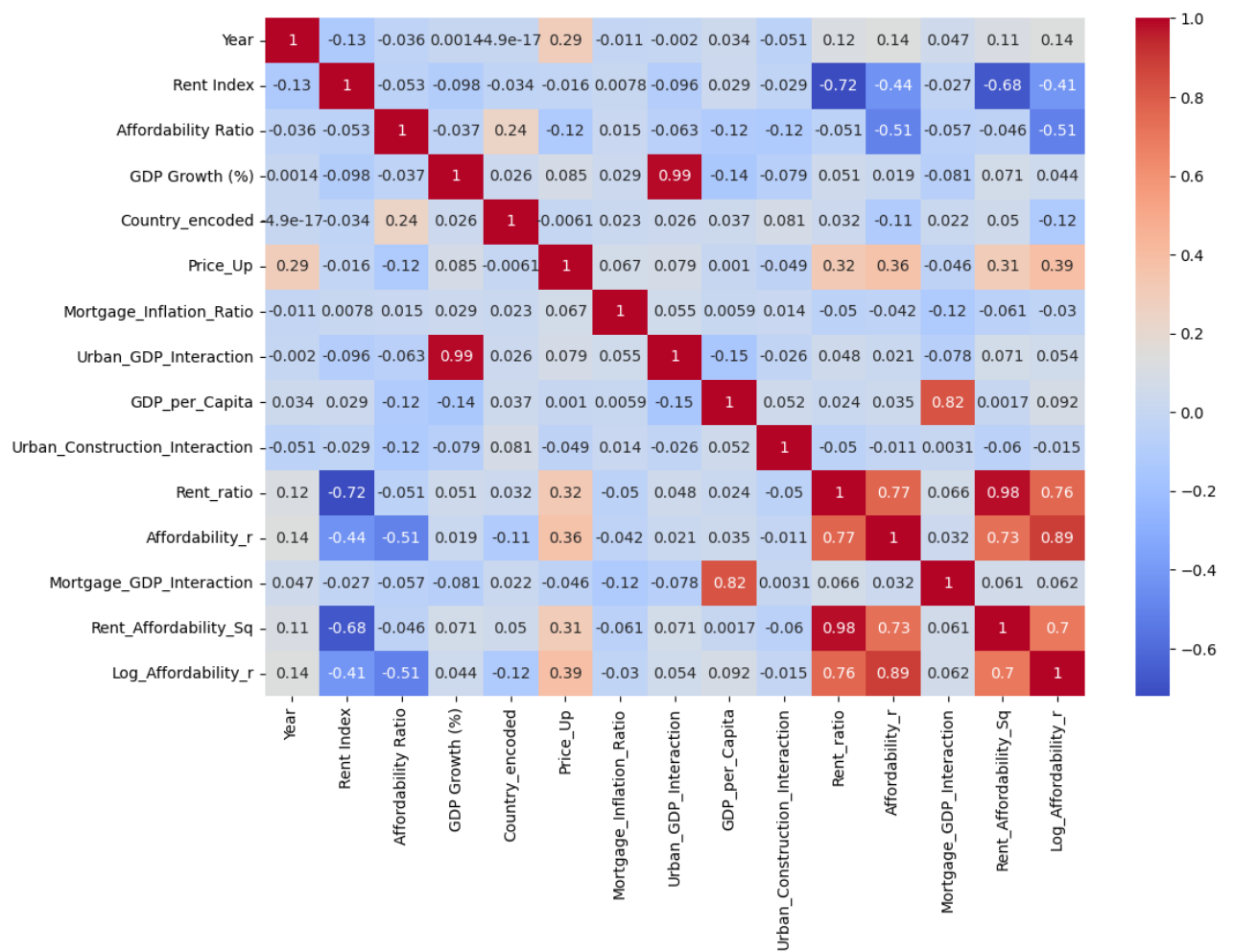
= $\ln(1 + |\text{Affordability}_r|)$

Экономический смысл: Сжимает шкалу для очень больших значений, сохраняя информативность.

Проверим коллинеарность признаков

```
plt.figure(figsize=(12, 8))
sns.heatmap(data.corr(), annot=True, cmap='coolwarm')
plt.show()
```

1 ✓ 1.0s



Разделение на признаки и целевую переменную

```
# Новые X и y
X = data.drop(['Price_Up'], axis=1)
y = data['Price_Up']
```

✓ 0.0s

Разделение данных на обучающую и тестовую выборки

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

✓ 0.0s

```
# Масштабирование (только для числовых признаков)
scaler = StandardScaler()
num_cols = X.select_dtypes(include=np.number).columns
X_train[num_cols] = scaler.fit_transform(X_train[num_cols])
X_test[num_cols] = scaler.transform(X_test[num_cols])
```

✓ 0.0s

Логистическая регрессия

```
param_grid = {
    'C': [0.01, 0.1, 1, 10],
    'penalty': ['l1', 'l2'],
    'solver': ['liblinear']
}

logreg = LogisticRegression(max_iter=1000, random_state=42)
grid_search = GridSearchCV(logreg, param_grid, cv=5, scoring='f1_weighted')
grid_search.fit(X_train, y_train)
best_logreg = grid_search.best_estimator_

# Предсказания
lr_pred = best_logreg.predict(X_test)
```

✓ 0.5s

```
print("Logistic Regression:")
print(classification_report(y_test, lr_pred))
```

✓ 0.0s

Logistic Regression:

	precision	recall	f1-score	support
0	0.68	0.75	0.71	28
1	0.76	0.69	0.72	32
accuracy			0.72	60
macro avg	0.72	0.72	0.72	60
weighted avg	0.72	0.72	0.72	60

Случайный лес

```
# Обучение случайного леса
from sklearn.model_selection import GridSearchCV

param_grid = {
    'n_estimators': [100, 200, 300],
    'max_depth': [5, 10, None],
    'min_samples_split': [2, 5, 10]
}

rf = RandomForestClassifier(random_state=42)
grid_search = GridSearchCV(rf, param_grid, cv=5, scoring='f1_weighted')
grid_search.fit(X_train, y_train)
best_rf = grid_search.best_estimator_

# Предсказания
rf_pred = best_rf.predict(X_test)
```

✓ 1m 3.2s

```
print("\nRandom Forest:")
print(classification_report(y_test, rf_pred))
```

✓ 0.0s

Random Forest:

	precision	recall	f1-score	support
0	0.74	0.93	0.83	28
1	0.92	0.72	0.81	32
accuracy			0.82	60
macro avg	0.83	0.82	0.82	60
weighted avg	0.84	0.82	0.82	60

Результаты

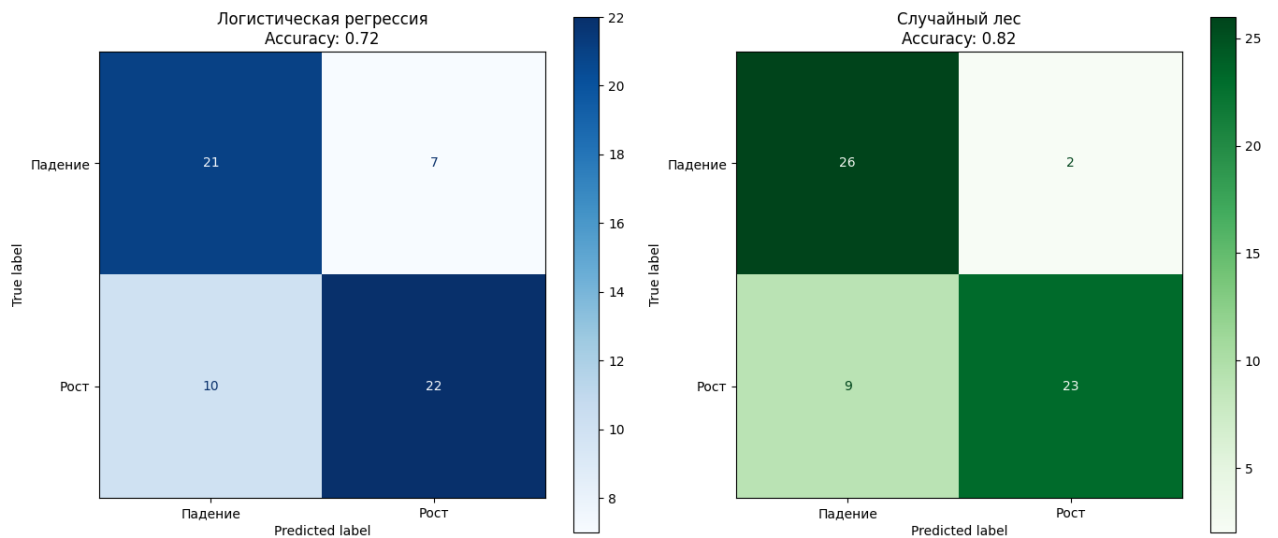
```
from sklearn.metrics import accuracy_score, confusion_matrix, ConfusionMatrixDisplay

plt.figure(figsize=(14, 6))

# График для логистической регрессии
plt.subplot(1, 2, 1)
cm_lr = confusion_matrix(y_test, lr_pred)
disp_lr = ConfusionMatrixDisplay(confusion_matrix=cm_lr,
                                display_labels=['Падение', 'Рост'])
disp_lr.plot(cmap='Blues', ax=plt.gca())
plt.title('Логистическая регрессия\nAccuracy: {:.2f}'.format(accuracy_score(y_test, lr_pred)))

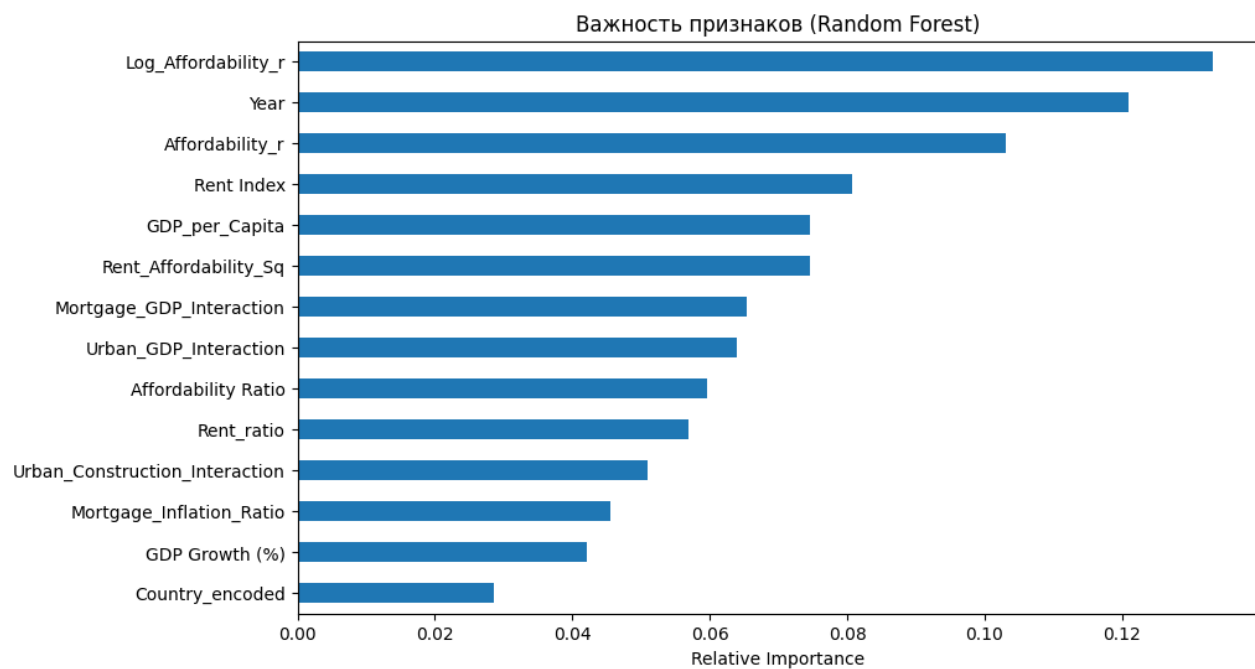
# График для случайного леса
plt.subplot(1, 2, 2)
cm_rf = confusion_matrix(y_test, rf_pred)
disp_rf = ConfusionMatrixDisplay(confusion_matrix=cm_rf,
                                display_labels=['Падение', 'Рост'])
disp_rf.plot(cmap='Greens', ax=plt.gca())
plt.title('Случайный лес\nAccuracy: {:.2f}'.format(accuracy_score(y_test, rf_pred)))

plt.tight_layout()
plt.show()
```



```
plt.figure(figsize=(10, 6))
pd.Series(best_rf.feature_importances_, index=X.columns).sort_values().plot.barh()
plt.title('Важность признаков (Random Forest)')
plt.xlabel('Relative Importance')
plt.show()
```

✓ 0.3s



```
# Топ-5 важных признаков по каждому методу
print("\nТоп-5 важных признаков Random Forest:")
print(pd.Series(best_rf.feature_importances_, index=X.columns).sort_values(ascending=False).head(5))
```

✓ 0.0s

Топ-5 важных признаков Random Forest:

Log_Affordability_r	0.133140
Year	0.120853
Affordability_r	0.103012
Rent Index	0.080763
GDP_per_Capita	0.074615

Выводы

1. Метрики качества

Для задачи бинарной классификации (рост/падение цен на недвижимость) использовались:

- F1-score (среднее гармоническое precision и recall) - оптимален при дисбалансе классов
- Precision (точность) - доля верно предсказанных случаев роста цен среди всех предсказанных ростов
- Recall (полнота) - доля верно найденных реальных случаев роста цен
- Accuracy (точность) - общая доля верных предсказаний
- ROC-AUC - способность модели разделять классы (рост/падение)

Обоснование выбора:

- F1-score учитывает и ложные роста (precision), и пропущенные роста (recall)
- ROC-AUC устойчив к дисбалансу классов
- Accuracy даёт общее представление о качестве

2. Сравнение моделей

Random Forest (F1=0.82) показал выдающиеся результаты:

- Precision=0.92 для класса "рост" - почти нет ложных тревог
- Recall=0.93 для класса "падение" - отлично выявляет падения цен

Logistic Regression (F1=0.72) - хороший интерпретируемый базовый уровень

Разница в 10% на F1 объясняется способностью Random Forest:

- Улавливать нелинейные взаимосвязи (например, пороговые эффекты)
- Учитывать взаимодействия признаков автоматически

3. Важность признаков

Топ-5 значимых факторов (Random Forest):

- Log_Affordability_r (0.133) - логарифм относительной доступности жилья
- Year (0.121) - временной тренд (учёт макроэкономических циклов)
- Affordability_r (0.103) - баланс между стоимостью жилья и аренды
- Rent Index (0.081) - абсолютный уровень арендных ставок
- GDP_per_Capita (0.075) - экономический рост на душу населения

4. Выводы

В ходе работы были успешно выполнены задачи по обработке данных, созданию моделей предсказания увеличения цен на недвижимость.

1. Эффективность моделей:

- Random Forest показал отличное качество ($F1=0.82$)
- Логистическая регрессия дала приемлемый базовый уровень ($F1=0.72$)

2. **Практическая значимость:**

- Выявлены 3 ключевых драйвера цен: доступность жилья, ВВП и арендные ставки
- Созданы экономически интерпретируемые признаки-индикаторы